
AcquisitionSynthesis: Targeted Data Generation using Acquisition Functions

Ishika Agarwal^{*1}, Sofia Stoica^{*1}, Emre Can Acikgoz¹,
 Pradeep Natarajan², Mahdi Namazifar², Jiaqi Ma¹, Dilek Hakkani-Tür¹
 University of Illinois Urbana-Champaign¹, Amazon²
 Correspondence to {ishikaa2, sstoica2}@illinois.edu

Abstract

Data quality remains a critical bottleneck in developing capable, competitive models. Researchers have explored many ways to generate top quality samples. Some works rely on rejection sampling: generating lots of synthetic samples and filtering out low-quality samples. Other works rely on larger or closed-source models to extract model weaknesses, necessary skills, or a curriculum off of which to base data generation. These works have one common limitation: there is no quantitative approach to measure the impact of the generated samples on the downstream learner. Active learning literature provides exactly this, in the form of acquisition functions. Acquisition functions measure the informativeness and/or influence of data, providing interpretable, model-centric signals. Inspired by this, we propose AcquisitionSynthesis: using acquisition functions as reward models to train language models to generate higher-quality synthetic data. We conduct experiments on classic verifiable tasks of math, medical question-answering, and coding. Our experimental results indicate that (1) student models trained with AcquisitionSynthesis data achieve good performance on in-distribution tasks (2-7% gain) and is more robust to catastrophic forgetting, and (2) AcquisitionSynthesis models can generate data for other models and for low-to-high resource training paradigms. By leveraging acquisition rewards, we seek to demonstrate a principled path toward model-aware self-improvement that surpasses static datasets.

1 Introduction

Despite data quality being a crucial driver to Large Language Model (LLM) progress, the guidelines for curating high quality data are not standardized or clear [Mehri et al., 2025, Koh and Liang, 2020, Li et al., 2025, Djuhera et al., 2025, Kim et al., 2025]. The current literature on data curation for LLMs is structured in two: (1) data selection, involving sampling a subset of data from a large set of given training data, and (2) synthetic data generation, involving generating data using LLMs based on given guidelines (either a set of skills/weaknesses/feedback [Sun et al., 2025, He et al., 2026, Mehri et al., 2025] or a seed set of training data [Sudalairaj et al., 2024, Jung et al., 2025, Cheng et al., 2025]). Our work focuses on **synthetic data generation**.

In this field, there are two general focuses: ensuring (1) **high quality data**, which requires generating large pools of synthetic data and applying rejection sampling to filter out low-quality samples [Wang et al., 2023, Jung et al., 2025, Ge et al., 2025], (2) **model-centric data**, which relies on languages models (within multi-step pipelines) to identify model weaknesses and construct training curricula [Khan et al., 2025a, Liang et al., 2025]. These methods can use a lot of compute to generate-then-filter, and are not always interpretable because using a qualitative approach that cannot measure the

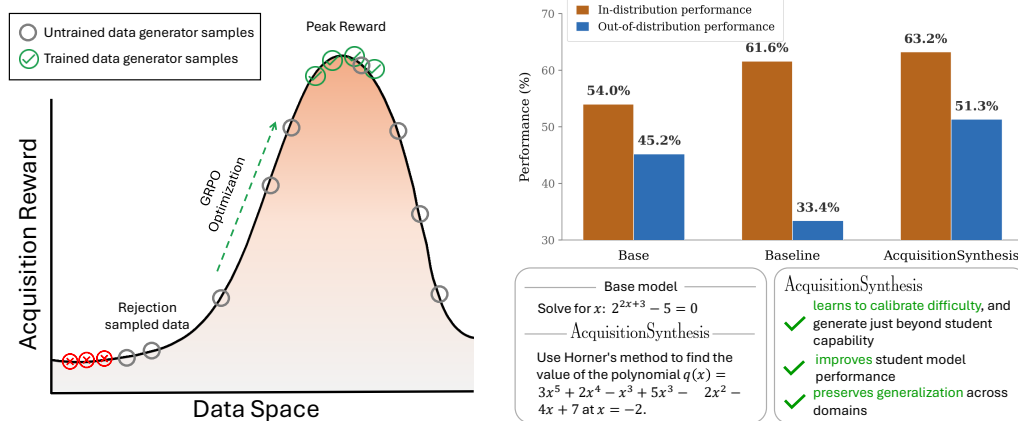


Figure 1: **Left:** AcquisitionSynthesis shifts generated samples toward high acquisition reward regions of the data space. Grey circles: samples from an untrained generator, scattered with varying rewards. Red x's: samples rejected by rejection sampling due to low reward. Green checks: samples from a trained AcquisitionSynthesis generator, concentrated near peak reward. **Right:** Plotted is the average in-distribution and out-of-distribution performance across tasks. While baselines improve in-distribution performance at the cost of OOD generalization, AcquisitionSynthesis improves both.

downstream impact of generated data provides no justification for why a given sample is useful. More importantly, despite the successes of these works, a fundamental question still remains unexplored:

Can simple, principled notions of data utility guide LLMs to generate better data?

The idea of data utility has been studied extensively in active learning [Settles, 2012].

Active learning, which involves selecting the most informative unlabeled samples for oracle annotation to reduce labeling cost while maximizing learning signal [Settles, 2012], has helped define simple and robust methods for measuring data utility called *acquisition functions*. In our work, we introduce AcquisitionSynthesis— a framework to train our models using Group Relative Policy Optimization (GRPO) Shao et al. [2024] with rewards inspired by acquisition functions to generate directly-usable, high quality data.

Figure 1(Left) illustrates our central finding: acquisition functions, when used as RL rewards, are simple yet surprisingly effective. By optimizing for these rewards, the generator learns to directly produce samples that are informative for a student model¹. Figure 1(Right) overviews the effects of AcquisitionSynthesis: acquisition rewards can be used to assess a student model’s weaknesses and adjust the data generation to push the student’s knowledge just beyond what it already knows.

Overview In AcquisitionSynthesis, to define data utility, we use five complementary acquisition rewards motivated by active learning literature that capture different dimensions of data quality: Confidence [Dredze and Crammer, 2008, Qi et al., 2026], Proximity [Xia et al., 2024], Gradient Magnitude [Ash et al., 2020, Gul et al., 2024], Diversity [Wang et al., 2017, Jung et al., 2025], and Answer Variance [Diao et al., 2024, Kee et al., 2018, Cao and Tsang, 2021]. We test AcquisitionSynthesis on verifiable tasks such as math, medical QA, and coding; by using AcquisitionSynthesis models to construct a dataset, we can train a student model on that dataset to determine the quality of the data generator (see Figure 2). We release our code here.

Our analyses uncover the following:

1. Student models trained on AcquisitionSynthesis data improve by $\sim 2\text{-}7\%$, with higher gains for bigger models (Section 5.1).

¹Unless stated otherwise, the student model is usually an instantiation of the AcquisitionSynthesis model’s untrained version.

2. Student models trained with AcquisitionSynthesis suffer less from catastrophic forgetting compared to baselines, and can even improve performance in out-of-distribution settings by 3% (Section 5.2).
3. For larger data-generator models, AcquisitionSynthesis trains stronger generators that generalize across model families and sizes (Section 5.3).
4. AcquisitionSynthesis generates data suitable for various training paradigms (Section 5.4).

2 Related Work

Learning Signals. Quantifying what makes a training sample “useful” has a long history in active learning [Settles, 2012], where *acquisition functions* guide the selection of which unlabeled samples to annotate. These acquisition functions use signals from geometry-based heuristics [Kang et al., 2025, Sener and Savarese, 2018, Mirzasoleiman et al., 2020] to uncertainty or margin-based sampling [Balcan et al., 2007, Lakshminarayanan et al., 2017]. Recent work has adapted these ideas to LLMs: [Agarwal et al., 2025, Rouzegar and Makrehchi, 2024] use model uncertainty, [Jung et al., 2025, Zhang et al., 2026] design diversity-based sampling methods, and Bayer et al. [2026] employ LLM-as-a-Judge scores as a quality proxy. However, all of these works operate in the *data selection* realm, where there exists a large pool of data already. In our paper, we build upon these works to show that language models can optimize data generation when given such a data quality reward.

Synthetic Data Generation. There are a variety of use cases that require models to generate their own training data: when there isn’t enough data, when the data has to be targeted for a particular domain, or the data has to adhere to certain stylistic guidelines. Self-Instruct [Wang et al., 2023] was one of the first papers that showed that language models can generate coherent samples and, indeed, improve language model capability. Since then, the focus of data generation has shifted to targeted data generation, focusing on (1) complexity [Xu et al., 2025], (2) persona-driven diversity [Ge et al., 2025], and (3) addressing model weaknesses [Khan et al., 2025a]. We reformulate data generation as an optimization problem (where GRPO [Shao et al., 2024] is the optimization mechanism), in which the acquisition reward formulation is our core contribution: a set of principled, interpretable objectives (grounded in active learning theory) that give the generator a quantitative target for what constitutes useful training data.

3 Methodology

Motivated by the active learning literature, we identify five acquisition function formulations to act as reward functions. Each of them are adapted for language models/transformers. Figure 2 contains an illustration of our evaluation framework. The data generator (i.e., AcquisitionSynthesis) model is trained using 500 samples –these samples are used as in-context learning examples (see Appendix B for more details)– during training, the AcquisitionSynthesis model generates a data sample which is fed to the student model. To clarify, the AcquisitionSynthesis model and student model are two instantiations of the same model. The acquisition reward is calculated with respect to the student model, in one of the ways below, and the reward is sent back to the AcquisitionSynthesis model to update its parameters.

Here are the different acquisition functions we use:

1. **Confidence:** this acquisition function rewards a model based on how *un*-confident the student model is on a generated data point. In active learning literature, this is also known as “uncertainty sampling” or “margin sampling” [Balcan et al., 2007]. We calculate confidence by the average difference in probability between the top-2 most probable tokens over the sequence², and assign an acquisition reward based on the inverse of the average³. This acquisition function incentivizes models to generate data points that they are most uncertain about Dredze and Crammer [2008], Qi et al. [2026], in order to learn most about the task.

²Given a prompt P and response R (where $R_t^{(i)}$ is the i -th most probable token in the t -th spot), the LLM’s confidence is:

$$\text{conf}(P, R) = \frac{1}{|R|} \sum_{t=1}^{|R|} \left(p_{\theta}(R_t^{(1)} \mid P, R_{<t}) - p_{\theta}(R_t^{(2)} \mid P, R_{<t}) \right)$$

³Following $\text{conf}(P, R)$, the reward is $\frac{1}{\text{conf}(P, R)}$.

2. **Proximity:** we reward the model based on how semantically similar the generated data point is to the given task’s training data to avoid irrelevant samples [Xia et al., 2024]. We cluster the data using HDBSCAN [Malzer and Baum, 2020], a dynamic method of clustering that does not require specification of the number of clusters beforehand. The acquisition reward assigned is the cosine distance from the generated sample to the closest cluster center.
3. **Gradient Magnitude:** this acquisition function assigns a reward based on how significant the learning signal is [Ash et al., 2020, Gul et al., 2024], i.e., the Frobenius norm of the gradient induced by backpropagating on the student model given the generated data sample. This incentivizes the model to generate samples that “surprise” the student model and can achieve the highest learning signal from.
4. **Diversity:** we reward the model based on how semantically diverse the generated data point is to the 500 training samples to avoid mode collapse. This incentivizes the model to generate samples that are different from what it has already seen before [Wang et al., 2017, Jung et al., 2025]. It is the opposite of the Proximity reward: $1 - p$ where p is the proximity reward.
5. **Answer variance:** this acquisition function rewards a model based on the number of unique answers it outputs in $k = 8$ model inferences. In active learning literature, this is close to the “query-by-committee” sampling strategy [Kee et al., 2018, Burbidge et al., 2007]. We assign a higher reward if the student model is more confused. We cluster the responses using HDBSCAN, and report the number of clusters as the reward. This incentivizes the model to generate samples that the student model is uncertain how to answer [Diao et al., 2024, Kee et al., 2018, Cao and Tsang, 2021]. The motivation is similar to Confidence; however, Confidence measures token-level uncertainty while Answer Variance measures semantic-level uncertainty.

To ensure that our generated data is parseable during post-processing, we apply a format reward following previous RLVR works. The reward is 0 if the generated data is parseable (via HTML tags such as “question”, “reasoning”, and “answer”) and -1 if there are any errors. This reward is applied during all AcquisitionSynthesis training, but we also use this as an ablation in our experimentation, and call it **Format**.

3.1 Dataset Construction for Student Model

Since we ask the model to generate its own training labels, label reliability is an open research problem. Following prior work, we assign pseudo-labels based on agreement across multiple sampled outputs. In our open-ended tasks, where exact string matching is infeasible, we embed the N sampled responses with a sentence encoder (Qwen/Qwen3-Embedding-0.6B [Zhang et al., 2025], in particular because of its small size yet strong performance on MTEB [Muennighoff et al., 2022]), cluster them by cosine similarity, and treat the medoid of the largest cluster as the pseudo-label. In Appendix A, we run a small-scale experiment to see the effect of more reliable labeler (GPT-4o-mini) and see that we can get a 6% performance boost if we use better labels.

4 Experimental Evaluation

Figure 2 contains an illustration of our experimental setup. In Step 1, we train our AcquisitionSynthesis model with 500 seed samples of data (again, as ICL examples – see Appendix B). Next, we use the trained model to generate training data for a student model (Step 2). We generate 1,000 samples (and keep this consistent with all of our baselines). After training the student model with SFT on the generated samples, we evaluate the student model on a test set. We aim to achieve better student model performance than using the original curated datasets or generated data from other methods. We use 6 A100 NVIDIA GPUs for performing GRPO training, which takes roughly 30-60 minutes for training AcquisitionSynthesis models.

Models. For main results, we use Qwen2.5-3B-Instruct (shorthand: Qwen 3B) and Llama-3.1-8B-Instruct (shorthand Llama 8B). To show the generalization of our method to various model sizes and architectures, we also use Qwen2.5-7B-Instruct (shorthand: Qwen 7B).

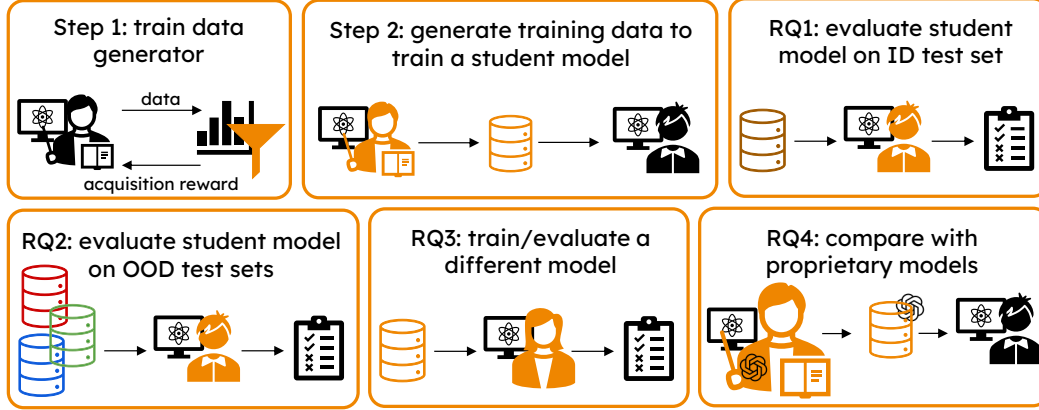


Figure 2: an illustration of our evaluation framework. Steps 1 and 2 describe our dataset generation while our evaluation for RQ’s 1-4 are illustrated. More details are introduced in Section 5. To clarify, we only use the generated dataset from AcquisitionSynthesis to train our student model.

Datasets. We evaluate on three types of datasets: math, medical, and coding. For math, we train on Numina (AI-M0/NuminaMath-CoT) [LI et al., 2024] and evaluate on a held-out portion of Numina as well as AIME24 (HuggingFaceH4/aime_2024). For medical, we train on MedMCQA (openlifescienceai/medmcqa) [Pal et al., 2022] and evaluate on a held-out test set of MedMCQA as well as PubMedQA (qiaojin/PubMedQA) [Jin et al., 2019]. Finally, for coding, we only use CodeForces (open-r1/codeforces-cots) [Penedo et al., 2025] for evaluation.

Baselines We use the following data synthesis baselines to test the robustness of our method:

1. **Original:** we use the frozen model to generate data samples, with no guidance.
2. **DataEnvGym** [Khan et al., 2025b]: given data for a particular task, a student model is evaluated on the data set and the samples on which it performed poorly are collected. A data generator model is then used to identify the skills required to solve those samples, and generate new question-answer pairs to target those weaknesses. Using this baseline, we aim to show the benefit of using more principled and amortized approaches for determining weaknesses and generating samples to target weaknesses.
3. **Prismatic Synthesis** [Jung et al., 2025]: this method curates diverse data by generating 100,000 data samples, clustering them by their gradient vectors, and discarding the samples that belong to dense gradient clusters. Using this baseline, we aim to show the benefits of removing post-hoc data filtering.

We also evaluate on the following two data-selection baselines to show why synthesis can produce higher-quality samples than pruning existing training data:

1. **Random:** we randomly select 1000 data samples to use as training data for the student model.
2. **Filtered:** we use our acquisition functions in a pool-based active learning set up. We score each data sample in the pool using the acquisition functions, normalize the scores, and average across the five acquisition functions to get a final ranking. We prune out the top 1000 data samples to use as training data.

5 Results

We break our results and analysis into four research questions (RQs) (see Figure 2 for illustrations of our RQ evaluations) regarding in-distribution vs out-of-distribution performance, student model transfer, and student model training paradigms. We also analyze the token efficiency of AcquisitionSynthesis models. The details are in the following sections.

5.1 RQ1: how well does AcquisitionSynthesis generate data?

Category	Method	Qwen 3B			Llama 8B			Qwen 7B		
		Numina	MedMCQA	Avg.	Numina	MedMCQA	Avg.	Numina	MedMCQA	Avg.
	Untrained model	28.3	55.1	41.7	32.2	75.8	54.0	29.3	67.1	48.2
Selection	Random	28.8 $\uparrow 0.5$	56.4 $\uparrow 1.3$	42.6 $\uparrow 0.9$	43.7 $\uparrow 11.5$	82.4 $\uparrow 6.6$	63.0 $\uparrow 9.0$	22.9 $\downarrow 6.4$	55.3 $\downarrow 11.8$	37.9 $\downarrow 10.3$
	Filtered	29.2 $\uparrow 0.9$	54.3 $\downarrow 0.8$	41.7 $\uparrow 0.0$	45.2 $\uparrow 13.0$	79.3 $\uparrow 3.5$	62.2 $\uparrow 8.2$	34.8 $\uparrow 5.5$	75.4 $\uparrow 8.3$	55.1 $\uparrow 6.9$
Synthesis	Original	29.4 $\uparrow 1.1$	55.5 $\uparrow 0.4$	42.4 $\uparrow 0.7$	44.5 $\uparrow 12.3$	78.8 $\uparrow 3.0$	61.6 $\uparrow 7.6$	36.5 $\uparrow 7.2$	75.3 $\uparrow 8.2$	55.9 $\uparrow 7.7$
	DataEnvGym	32.3 $\uparrow 4.0$	55.1 $\downarrow 0.0$	43.7 $\uparrow 2.0$	26.8 $\downarrow 5.4$	44.4 $\downarrow 31.4$	35.6 $\downarrow 18.4$	17.8 $\downarrow 11.5$	76.2 $\uparrow 9.1$	47.0 $\downarrow 1.2$
	Prismatic Syn.	44.6 $\uparrow 16.3$	65.1 $\uparrow 10.0$	54.8 $\uparrow 13.1$	33.0 $\uparrow 0.8$	73.8 $\downarrow 2.0$	53.4 $\downarrow 0.6$	34.8 $\uparrow 5.5$	66.4 $\downarrow 0.7$	50.6 $\uparrow 2.4$
AS	Format	29.6 $\uparrow 1.3$	57.3 $\uparrow 2.2$	43.4 $\uparrow 1.7$	43.1 $\uparrow 10.9$	81.9 $\uparrow 6.1$	62.5 $\uparrow 8.5$	43.2 $\uparrow 13.9$	74.1 $\uparrow 7.0$	58.7 $\uparrow 10.45$
	Confidence	33.7 $\uparrow 5.4$	59.5 $\uparrow 4.4$	46.6 $\uparrow 4.9$	45.5 $\uparrow 13.3$	81.0 $\uparrow 5.2$	63.2 $\uparrow 9.2$	43.1 $\uparrow 13.8$	75.0 $\uparrow 7.9$	59.1 $\uparrow 10.9$
	Proximity	22.1 $\downarrow 6.2$	63.6 $\uparrow 8.5$	42.9 $\uparrow 1.2$	40.7 $\uparrow 8.5$	77.0 $\uparrow 1.2$	58.9 $\uparrow 4.9$	40.2 $\uparrow 10.9$	74.5 $\uparrow 7.4$	57.4 $\uparrow 9.2$
	Gradient Norm	29.7 $\uparrow 1.4$	58.6 $\uparrow 3.5$	44.2 $\uparrow 2.5$	43.7 $\uparrow 11.5$	80.9 $\uparrow 5.1$	62.3 $\uparrow 8.3$	42.5 $\uparrow 13.2$	70.5 $\uparrow 3.4$	56.5 $\uparrow 8.3$
	Diversity	24.7 $\downarrow 3.6$	58.0 $\uparrow 2.9$	41.4 $\downarrow 0.3$	42.8 $\uparrow 10.6$	77.8 $\uparrow 2.0$	60.3 $\uparrow 6.3$	45.2 $\uparrow 15.9$	75.2 $\uparrow 8.1$	60.2 $\uparrow 12.0$
	Answer Variance	30.5 $\uparrow 2.2$	55.1 $\downarrow 0.0$	42.8 $\uparrow 1.1$	43.2 $\uparrow 11.0$	80.3 $\uparrow 4.5$	61.7 $\uparrow 7.7$	43.4 $\uparrow 14.1$	75.3 $\uparrow 8.2$	59.4 $\uparrow 11.2$

Table 1: **In-distribution performance.** Absolute gains/losses are computed relative to the untrained model for each model and benchmark. ($\uparrow$) and ($\downarrow$) denote absolute gains and losses relative to the untrained model, respectively.

Table 1 contains our results for in-distribution performance, showing the performance of all our baselines and AcquisitionSynthesis variants, with Qwen 3B, Qwen 7B, and Llama 8B. On average, AcquisitionSynthesis improves model performance by 1.82% for Qwen and 7.47% for Llama. The data selection baselines achieve meager improvements – this might be because of the question difficulty; as the untrained model’s performance improves, the gains due to data selection also improve. Hence, there are two key limitations of data selection: (1) a large pool of questions is assumed, and (2) question difficulty does not always correlate to high learning signal. On the other hand, AcquisitionSynthesis and the other **data synthesis methods are able to calibrate the question difficulty** to ensure the model learns most effectively from its data.

Regarding the data synthesis methods, Original is an unreliable baseline as its performance is setting dependent: it is sometimes a strong baseline (Llama 8B for Numina), and other times it does not yield performance gains (Qwen 3B). DataEnvGym and Prismatic Synthesis are strong baselines – they directly target model weaknesses and are able to improve a model’s performance on the test set. Moreover, Prismatic Synthesis uses a small, proxy model to compute gradients in order to cluster and select diverse gradients. Following their setup, we use Qwen2.5-0.5B-Instruct, which could explain why Prismatic Synthesis works better on Qwen 3B than on Llama 8B. To conclude, **models trained with AcquisitionSynthesis data show competitive performance, no matter the model family or size.**

Evaluated on		Trained on Numina				Trained on MedMCQA			
		AIME	MedMCQA	CodeForces	Avg.	PubMedQA	Numina	CodeForces	Avg.
	Qwen 3B	43.3	55.1	51.5	50.0	28.5	28.3	51.5	36.1
Selection	Random	56.7 $\uparrow 13.3$	62.6 $\uparrow 7.4$	77.6 $\uparrow 26.1$	65.6 $\uparrow 15.6$	30.8 $\uparrow 2.3$	34.3 $\uparrow 6.0$	46.9 $\downarrow 4.6$	37.3 $\uparrow 1.2$
	Filtered	40.0 $\downarrow 3.3$	54.3 $\downarrow 0.8$	78.3 $\uparrow 26.8$	57.5 $\uparrow 7.5$	12.5 $\downarrow 16.0$	29.5 $\uparrow 1.2$	23.9 $\downarrow 27.6$	22.0 $\downarrow 14.1$
Synthesis	Original	40.0 $\downarrow 3.3$	62.1 $\uparrow 7.0$	40.1 $\downarrow 11.4$	47.4 $\downarrow 2.6$	31.3 $\uparrow 2.8$	40.0 $\uparrow 11.7$	40.0 $\downarrow 11.5$	37.1 $\uparrow 1.0$
	DataEnvGym	30.0 $\downarrow 13.3$	65.3 $\uparrow 10.2$	41.5 $\downarrow 10.0$	45.6 $\downarrow 4.4$	45.5 $\uparrow 17.0$	35.7 $\uparrow 7.4$	55.7 $\uparrow 4.2$	45.6 $\uparrow 9.5$
	Prismatic Syn.	33.3 $\downarrow 10.0$	60.9 $\uparrow 5.8$	23.6 $\downarrow 27.9$	39.3 $\downarrow 10.7$	28.2 $\downarrow 0.3$	48.1 $\uparrow 19.8$	30.9 $\downarrow 20.6$	35.7 $\downarrow 0.4$
AS	Format	40.0 $\downarrow 3.3$	58.3 $\uparrow 3.2$	49.7 $\downarrow 1.8$	49.4 $\downarrow 0.6$	26.2 $\downarrow 2.3$	34.2 $\uparrow 5.9$	38.5 $\downarrow 13.0$	33.0 $\downarrow 3.1$
	Confidence	60.0 $\uparrow 16.7$	63.1 $\uparrow 8.0$	63.9 $\uparrow 12.4$	62.4 $\uparrow 12.4$	38.5 $\uparrow 10.0$	38.1 $\uparrow 9.8$	53.4 $\uparrow 1.9$	43.3 $\uparrow 7.2$
	Proximity	26.7 $\downarrow 16.7$	48.4 $\downarrow 6.8$	50.5 $\downarrow 1.0$	41.9 $\downarrow 8.1$	32.5 $\uparrow 4.0$	43.2 $\uparrow 14.9$	54.0 $\uparrow 2.5$	43.2 $\uparrow 7.1$
	Gradient Norm	43.3 $\uparrow 0.0$	60.6 $\uparrow 5.5$	53.7 $\uparrow 2.2$	52.6 $\uparrow 2.6$	32.8 $\uparrow 4.3$	36.4 $\uparrow 8.1$	45.8 $\downarrow 5.7$	38.3 $\uparrow 2.2$
	Diversity	40.0 $\downarrow 3.3$	51.2 $\downarrow 3.9$	70.6 $\uparrow 19.1$	53.9 $\uparrow 4.0$	35.8 $\uparrow 7.3$	38.1 $\uparrow 9.8$	67.6 $\uparrow 16.1$	47.1 $\uparrow 11.1$
	Answer Variance	40.0 $\downarrow 3.3$	61.3 $\uparrow 6.1$	55.0 $\uparrow 3.5$	52.1 $\uparrow 2.1$	30.3 $\uparrow 1.8$	38.4 $\uparrow 10.1$	52.3 $\uparrow 0.8$	40.3 $\uparrow 4.2$

Table 2: **Generalization results on Qwen.** This table shows the performance of student models trained on curated data from one distribution, but evaluated using other distributions (on the same domain, or different domains). For **Numina**, AIME is in the same domain, while MedMCQA and CodeForces are in different domains. Similarly, for **MedMCQA**, PubMedQA is in the same domain, while Numina and CodeForces are in different domains. ($\uparrow$) and ($\downarrow$) indicate absolute improvements and degradations relative to the untrained model, respectively.

5.2 RQ2: how do models trained with AcquisitionSynthesis data perform on OOD tasks?

To measure for catastrophic forgetting, we evaluate each trained student on other out-of-distribution datasets: one in the same domain, and two in different domains. To clarify, if a student were trained on Numina (domain: math), the same domain dataset would be AIME and the different domain datasets would be MedMCQA and CodeForces. Tables 2 and 3 contain those results.

Student models trained with the data selection baselines perform inconsistently in out-of-distribution settings, where Qwen 3B achieves a 2.56% increase, while Llama 8B achieves a 5.11% decrease in performance. Data selection assumes that the training distribution matches the testing distribution: our results indicate that when the distributions don’t match, the performance drops in OOD settings. Regarding the data synthesis baselines, even though we saw good performance for in-distribution tasks, the same performance gains do not transfer OOD. Collectively, the synthesis baselines degrades Qwen’s performance by 1.25% and Llama’s performance by 12.94%. On the other hand, AcquisitionSynthesis improves Qwen by 3.43% and Llama maintains its average performance (only a slight increase of 0.25%). These experiments show that **student models trained on AcquisitionSynthesis data are less prone to catastrophic forgetting, compared to baselines.**

Evaluated on		Trained on Numina				Trained on MedMCQA			
		AIME	MedMCQA	CodeForces	Avg.	PubMedQA	Numina	CodeForces	Avg.
	Llama 8B	26.7	75.8	48.7	50.4	39.0	32.2	48.7	40.0
Selection	Random	63.3 $\uparrow 36.7$	77.4 $\uparrow 1.5$	15.0 $\downarrow 33.7$	51.9 $\uparrow 1.5$	12.5 $\downarrow 26.5$	38.2 $\uparrow 6.0$	19.2 $\downarrow 29.5$	23.3 $\downarrow 16.7$
	Filtered	70.0 $\uparrow 43.3$	82.0 $\uparrow 6.2$	10.0 $\downarrow 38.7$	54.0 $\uparrow 3.6$	30.0 $\downarrow 9.0$	38.4 $\uparrow 6.2$	25.0 $\downarrow 23.7$	31.1 $\downarrow 8.8$
Synthesis	Original	23.3 $\downarrow 3.3$	41.9 $\downarrow 33.9$	47.7 $\downarrow 1.0$	37.6 $\downarrow 12.7$	36.5 $\downarrow 2.5$	27.4 $\downarrow 4.9$	25.9 $\downarrow 22.8$	29.9 $\downarrow 10.1$
	DataEnvGym	23.3 $\downarrow 3.3$	44.6 $\downarrow 31.3$	20.6 $\downarrow 28.1$	29.5 $\downarrow 20.9$	34.0 $\downarrow 5.0$	26.7 $\downarrow 5.5$	20.2 $\downarrow 28.5$	27.0 $\downarrow 13.0$
	Prismatic Syn.	13.3 $\downarrow 13.3$	62.8 $\downarrow 13.1$	23.6 $\downarrow 25.1$	33.2 $\downarrow 17.2$	36.3 $\downarrow 2.8$	28.4 $\downarrow 3.8$	32.5 $\downarrow 16.2$	32.4 $\downarrow 7.6$
AS	Format	60.0 $\uparrow 33.3$	81.9 $\uparrow 6.1$	45.8 $\downarrow 2.9$	62.6 $\uparrow 12.2$	43.8 $\uparrow 4.8$	36.6 $\uparrow 4.4$	40.0 $\downarrow 8.7$	40.1 $\uparrow 0.2$
	Confidence	30.0 $\uparrow 3.3$	81.0 $\uparrow 5.2$	41.3 $\downarrow 7.4$	50.8 $\uparrow 0.4$	38.8 $\downarrow 0.3$	39.4 $\uparrow 7.2$	27.5 $\downarrow 12.2$	35.2 $\downarrow 4.8$
	Proximity	26.7 $\uparrow 0.0$	77.0 $\uparrow 1.2$	57.5 $\uparrow 8.8$	53.7 $\uparrow 3.3$	40.0 $\uparrow 1.0$	43.3 $\uparrow 11.1$	45.7 $\downarrow 3.0$	43.0 $\uparrow 3.0$
	Gradient Norm	30.0 $\uparrow 3.3$	80.9 $\uparrow 5.0$	31.6 $\downarrow 17.1$	47.5 $\downarrow 2.9$	38.0 $\downarrow 1.0$	38.7 $\uparrow 6.5$	55.6 $\uparrow 6.9$	44.1 $\uparrow 4.1$
	Diversity	40.0 $\uparrow 13.3$	77.8 $\uparrow 2.0$	20.5 $\downarrow 28.2$	46.1 $\downarrow 4.3$	34.5 $\downarrow 4.5$	40.4 $\uparrow 8.2$	35.5 $\downarrow 13.2$	36.8 $\downarrow 3.2$
	Answer Variance	30.0 $\uparrow 3.3$	80.3 $\uparrow 4.4$	32.2 $\downarrow 16.5$	47.5 $\downarrow 2.9$	32.8 $\downarrow 6.3$	38.5 $\uparrow 6.3$	34.3 $\downarrow 14.4$	35.2 $\downarrow 4.8$

Table 3: **Generalization results on Llama.** It is similar to Table 2. ($\uparrow$) and ($\downarrow$) indicate absolute improvements and degradations relative to the untrained model, respectively.

Key Takeaway from RQ1 and RQ2

AcquisitionSynthesis generates data that is not only useful for in-domain learning, but also better aligned with the broader evaluation distribution. Furthermore, Confidence is the best general-purpose reward, and Diversity offers the best cost-to-OOD-gain tradeoff.

Why AcquisitionSynthesis Generalizes. To explain the OOD gains of AcquisitionSynthesis, we measure the distributional alignment between the generated training data and held-out benchmark data, computed as the average cosine similarity between their embeddings [Acikgoz et al., 2026]. Higher values indicate that synthesis covers the regions of input space encountered at test time. Table 4 shows that our Confidence variant attains the highest alignment on every benchmark compared to previous works. This suggests that uncertainty-guided generation better covers the regions of the test distribution where the model is likely to make mistakes, rather than only producing diverse or weakness-targeted samples [Huang et al., 2023]. These results provide one explanation for the stronger generalization observed in RQ2: AcquisitionSynthesis generates data that is not only useful for in-domain learning, but also better aligned with the broader evaluation distribution.

Model	AIME	MedMCQA	CodeForces	Avg.
DataEnvGym [Khan et al., 2025a]	0.558	0.464	0.549	0.524
Prismatic Syn. [Jung et al., 2025]	0.566	0.456	0.557	0.526
Confidence (Ours)	0.575	0.497	0.560	0.544

Table 4: Distributional coverage of generated data on OOD benchmarks.

5.3 RQ3: can AcquisitionSynthesis models generalize and generate data for other models?

In production settings, we can not guarantee that the student model will have the same architecture or size as the generator model. Thus, this research question explores our trained AcquisitionSynthesis

Llama 8B trained on		Numina				MedMCQA			
Evaluated on		Llama 3B	Qwen 3B	Qwen 7B	Avg.	Llama 3B	Qwen 3B	Qwen 7B	Avg.
Untrained		33.57	28.27	29.27	30.37	49.03	55.15	67.13	57.10
Synthesis	Original	41.85 $\uparrow 8.28$	41.06 $\uparrow 12.79$	44.96 $\uparrow 15.69$	42.62 $\uparrow 12.25$	65.95 $\uparrow 16.92$	61.45 $\uparrow 6.30$	61.84 $\downarrow 5.29$	63.08 $\uparrow 5.98$
	DataEnvGym	24.67 $\downarrow 8.90$	24.48 $\downarrow 3.79$	28.47 $\downarrow 0.80$	25.87 $\downarrow 4.50$	22.96 $\downarrow 26.07$	49.97 $\downarrow 5.18$	51.07 $\downarrow 16.06$	41.33 $\downarrow 15.77$
	Prismatic Syn.	19.88 $\downarrow 13.69$	21.87 $\downarrow 6.40$	42.76 $\uparrow 13.49$	28.17 $\downarrow 2.20$	75.60 $\uparrow 26.57$	60.48 $\uparrow 5.33$	70.35 $\uparrow 3.22$	68.81 $\uparrow 11.71$
AS	Format	41.36 $\uparrow 7.79$	45.05 $\uparrow 16.78$	45.05 $\uparrow 15.78$	43.82 $\uparrow 13.45$	68.67 $\uparrow 19.64$	62.86 $\uparrow 7.71$	64.37 $\downarrow 2.76$	65.30 $\uparrow 8.20$
	Confidence	41.05 $\uparrow 7.48$	43.26 $\uparrow 14.99$	47.85 $\uparrow 18.58$	44.05 $\uparrow 13.68$	65.62 $\uparrow 16.59$	57.79 $\uparrow 2.64$	61.83 $\downarrow 5.30$	61.75 $\uparrow 4.65$
	Proximity	36.16 $\uparrow 2.59$	41.26 $\uparrow 12.99$	47.25 $\uparrow 17.98$	41.56 $\uparrow 11.19$	69.11 $\uparrow 20.08$	54.57 $\downarrow 0.58$	69.44 $\uparrow 2.31$	64.37 $\uparrow 7.27$
	Gradient Norm	40.26 $\uparrow 6.69$	42.76 $\uparrow 14.49$	46.85 $\uparrow 17.58$	43.29 $\uparrow 12.92$	64.86 $\uparrow 15.83$	57.88 $\uparrow 2.73$	57.59 $\uparrow 9.54$	60.11 $\uparrow 3.01$
	Diversity	39.36 $\uparrow 5.79$	40.76 $\uparrow 12.49$	47.75 $\uparrow 18.48$	42.62 $\uparrow 12.25$	66.06 $\uparrow 17.03$	59.57 $\uparrow 4.42$	65.25 $\downarrow 1.88$	63.63 $\uparrow 6.53$
Answer Variance		42.36 $\uparrow 8.79$	43.56 $\uparrow 15.29$	44.65 $\uparrow 15.38$	43.52 $\uparrow 13.15$	73.65 $\uparrow 24.62$	58.17 $\uparrow 3.02$	64.79 $\downarrow 2.34$	65.54 $\uparrow 8.44$

Table 5: **Model transfer results on Llama.** $\uparrow$ and $\downarrow$ indicate absolute improvements and degradations relative to the untrained model, respectively.

models’ ability to generate data for models of varying size and architectures. We train Qwen 3B, Qwen 7B, and meta-llama/Llama-3.2-3B-Instruct (Llama 3B) from data generated by Llama 8B. Table 5 contains the results. We don’t report results on the opposite setting (of training models with data generated by Qwen 3B) because Qwen 3B is too small of a model to distill data from – the notion of improving weak-to-strong model generalization is out-of-scope for our paper.

The data synthesis baselines are not able to generate data for other models: Original generalizes well on Llama 3B and Qwen 3B because that is directly knowledge distillation. DataEnvGym performs badly because the generated data is supposed to target a particular model’s weaknesses, and, thus, cannot generalize to other models. Prismatic Synthesis’s assumption of diverse data is slightly inconsistent: good for some domains, but not all. Meanwhile, **AcquisitionSynthesis methods maintain stable improvement across model families and sizes** – especially on datasets where untrained models do not perform well (in particular, math).

5.4 RQ4: how does student model performance vary when trained in low-to-high resource training methods?

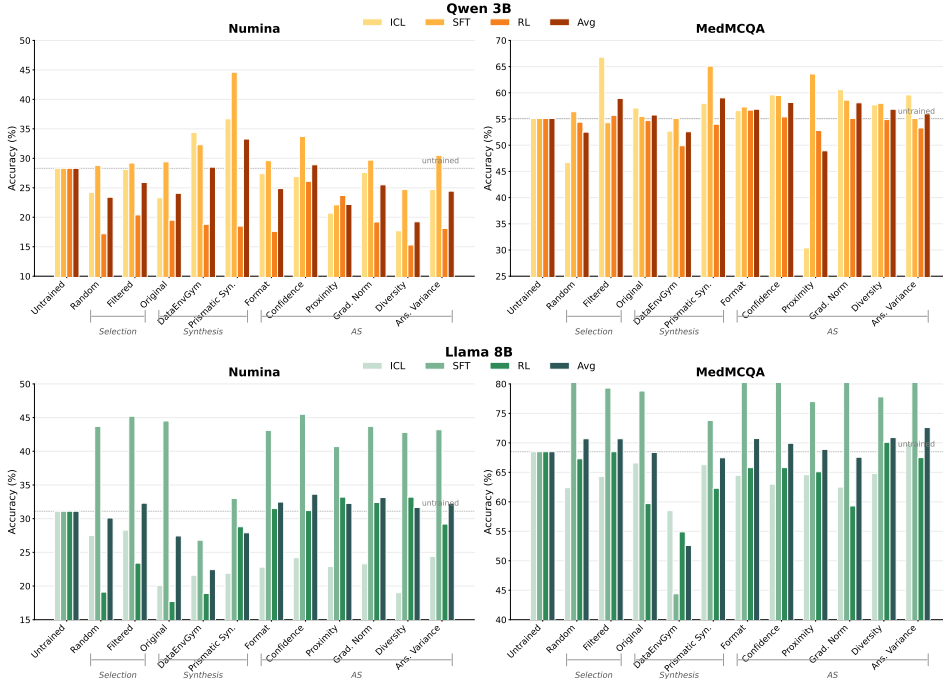


Figure 3: Performance of student models with varying training paradigms. We use the data selected/generated by each method and perform either ICL, SFT, or RL to obtain our student model.

In our work so far, we have used the curated (selected or generated) datasets as SFT datasets. But can we also use them in resource-constrained (in-context learning, or ICL) or resource-rich environments (reinforcement learning, or RL)? Figure 3 contains the results. Overall, we see SFT remains the strongest paradigm, but this is expected because (1) 3-billion parameter models might not be strong enough to use in-context examples effectively, and (2) we use an accuracy reward for RL training, which might be too sparse for a small model. For AcquisitionSynthesis, the direction of performance gains is mostly consistent across training paradigms: if the student model improves with SFT, it also improves with ICL and RL. This consistency is not guaranteed for baselines – for example, (1) Filtered improves substantially with ICL on MedMCQA but degrades with SFT, or (2) Prismatic Synthesis degrades with RL on Numina, despite strong SFT gains.

Our hypothesis for why AcquisitionSynthesis can adapt to any training paradigm is that each training paradigm requires data with different signals (ICL might require data with the maximum teachable content, SFT would benefit from large, un-noisy gradient signals, and RL is most effective with data that has dense learning signals). Our baselines optimize for a single axis of this: Prismatic Synthesis optimizes for diversity which is why it does well SFT; Random optimizes for coverage; DataEnvGym optimizes for model weaknesses, which provides inconsistent signals (SFT is the best training paradigm for Qwen 3B, but the worst for Llama 8B). AcquisitionSynthesis, on the other hand, provides one framework to generate data for any kind of training stage that necessitates any particular data characteristic. To summarize, AcquisitionSynthesis data can be reliably used to train models in resource-constrained (ICL) to resource-abundant (RL) environments without restructuring the data.

Key Takeaway from RQ3 and RQ4

Data generated from AcquisitionSynthesis models can be used flexibly to train student models of different families and sizes, and for any training paradigm (ICL, SFT, RL) reliably.

5.5 Discussion on token efficiency

During the data generation phase, we only apply one filtering pass: we discard any exact-match, duplicate questions. Due to this filtering, on average, Original discards 26.0% of generated data, DataEnvGym discards 76.7%, Prismatic Synthesis 65.2% (via its own rejection sampling), and AcquisitionSynthesis discards 11.1% of data. Overall, AcquisitionSynthesis’s reward design is able to efficiently generate a dataset by discarding 44.9% fewer tokens.

6 Conclusion

In this work, we propose AcquisitionSynthesis as a way to make synthetic data generation more interpretable. To improve the interpretability, we train models with five different acquisition functions as reward functions using GRPO, and use the resulting AcquisitionSynthesis models to generate data for student models. We make two major findings. Firstly, student models trained on AcquisitionSynthesis data achieve good performance in both in-distribution and out-of-distribution settings, achieving average performance gains of 4% compared to the baselines. Secondly, we see that AcquisitionSynthesis data can generalize and be used to train models of different architectures and of different sizes. Additionally, it can be used for any kind of training paradigm (ICL, SFT, and RL), which broadens the practical applicability of our method to settings where fine-tuning may not be feasible. To summarize the tradeoffs between acquisition rewards, Confidence is the best general-purpose reward, and Diversity offers the best cost-to-OOD-gain tradeoff. Overall, our work shows that using acquisition rewards to train data generators teaches models what good data is and how to optimally generate good data. In the future, we hope to further refine AcquisitionSynthesis to be a Universal AcquisitionSynthesis model: one model to generate data for all domains, with one unified notion of data utility. This can help us create a self-evolving loop where models generate data for themselves, teach themselves skills, and automatically update their own curriculum.

Future Work Our work serves as a proof of concept that data generation models can be optimized, and introduces a framework for evaluating them. Future directions include: (1) combining acquisition rewards linearly or polynomially, (2) optimizing the AcquisitionSynthesis training data mixture, and (3) testing our framework on non-verifiable tasks such as instruction following.

References

- Emre Can Acikgoz, Cheng Qian, Jonas Hübötter, Heng Ji, Dilek Hakkani-Tür, and Gokhan Tur. Tool-r0: Self-evolving llm agents for tool-learning from zero data. *arXiv preprint arXiv:2602.21320*, 2026.
- Ishika Agarwal, Krishnateja Killamsetty, Lucian Popa, and Marina Danilevsky. Delift: Data efficient language model instruction fine tuning, 2025. URL <https://arxiv.org/abs/2411.04425>.
- Jordan T. Ash, Chicheng Zhang, Akshay Krishnamurthy, John Langford, and Alekh Agarwal. Deep batch active learning by diverse, uncertain gradient lower bounds, 2020. URL <https://arxiv.org/abs/1906.03671>.
- Maria-Florina Balcan, Andrei Broder, and Tong Zhang. Margin based active learning. In *International Conference on Computational Learning Theory*, pages 35–50. Springer, 2007.
- Markus Bayer, Justin Lutz, and Christian Reuter. Activellm: Large language model-based active learning for textual few-shot scenarios. *Transactions of the Association for Computational Linguistics*, 14:1–22, 2026. ISSN 2307-387X. doi: 10.1162/tacl.a.63. URL <http://dx.doi.org/10.1162/TACL.a.63>.
- Robert Burbidge, Jem J Rowland, and Ross D King. Active learning for regression based on query by committee. In *International conference on intelligent data engineering and automated learning*, pages 209–218. Springer, 2007.
- Xiaofeng Cao and Ivor W. Tsang. Bayesian active learning by disagreements: A geometric perspective, 2021. URL <https://arxiv.org/abs/2105.02543>.
- Kellen Tan Cheng, Anna Lisa Gentile, Chad DeLuca, and Guang-Jie Ren. STAR: Self-automated back-querying for production data generation. In Kentaro Inui, Sakriani Sakti, Haofen Wang, Derek F. Wong, Pushpak Bhattacharyya, Biplab Banerjee, Asif Ekbal, Tanmoy Chakraborty, and Dharendra Pratap Singh, editors, *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*, pages 133–148, Mumbai, India, December 2025. The Asian Federation of Natural Language Processing and The Association for Computational Linguistics. ISBN 979-8-89176-298-5. doi: 10.18653/v1/2025.ijcnlp-long.9. URL <https://aclanthology.org/2025.ijcnlp-long.9/>.
- Shizhe Diao, Pengcheng Wang, Yong Lin, Rui Pan, Xiang Liu, and Tong Zhang. Active prompting with chain-of-thought for large language models, 2024. URL <https://arxiv.org/abs/2302.12246>.
- Aladin Djuhera, Swanand Ravindra Kadhe, Syed Zawad, Farhan Ahmed, Heiko Ludwig, and Holger Boche. Fixing it in post: A comparative study of llm post-training data quality and model performance, 2025. URL <https://arxiv.org/abs/2506.06522>.
- Mark Dredze and Koby Crammer. Active learning with confidence. In *Proceedings of ACL-08: HLT, Short Papers*, pages 233–236, 2008.
- Tao Ge, Xin Chan, Xiaoyang Wang, Dian Yu, Haitao Mi, and Dong Yu. Scaling synthetic data creation with 1,000,000,000 personas, 2025. URL <https://arxiv.org/abs/2406.20094>.
- Shreen Gul, Mohamed Elmahallawy, Sanjay Madria, and Ardhendu Tripathy. Lplgrad: Optimizing active learning through gradient norm sample selection and auxiliary model training, 2024. URL <https://arxiv.org/abs/2411.15217>.
- Yinghui He, Abhishek Panigrahi, Yong Lin, and Sanjeev Arora. STAT: Skill-targeted adaptive training. In *First Workshop on CogInterp: Interpreting Cognition in Deep Learning Models*, 2026. URL <https://openreview.net/forum?id=HUIJRX1z5Rh>.
- Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1051–1068, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.67. URL <https://aclanthology.org/2023.emnlp-main.67/>.

- Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. Pubmedqa: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, 2019.
- Jaehun Jung, Seungju Han, Ximing Lu, Skyler Hallinan, David Acuna, Shrimai Prabhumoye, Mostafa Patwary, Mohammad Shoeybi, Bryan Catanzaro, and Yejin Choi. Prismatic synthesis: Gradient-based data diversification boosts generalization in llm reasoning, 2025. URL <https://arxiv.org/abs/2505.20161>.
- Seongjae Kang, Dong Bok Lee, Hyungjoon Jang, Dongseop Kim, and Sung Ju Hwang. Pcoreset: Effective active learning through knowledge distillation from vision-language models, 2025. URL <https://arxiv.org/abs/2506.00910>.
- Seho Kee, Enrique Del Castillo, and George Runger. Query-by-committee improvement with diversity and density in batch active learning. *Information Sciences*, 454:401–418, 2018.
- Zaid Khan, Elias Stengel-Eskin, Jaemin Cho, and Mohit Bansal. Dataenvgym: Data generation agents in teacher environments with student feedback. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL <https://openreview.net/forum?id=00SnKBGTsz>.
- Zaid Khan, Elias Stengel-Eskin, Jaemin Cho, and Mohit Bansal. Dataenvgym: Data generation agents in teacher environments with student feedback, 2025b. URL <https://arxiv.org/abs/2410.06215>.
- Seungone Kim, Juyoung Suk, Xiang Yue, Vijay Viswanathan, Seongyun Lee, Yizhong Wang, Kiril Gashteovski, Carolin Lawrence, Sean Welleck, and Graham Neubig. Evaluating language models as synthetic data generators. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6385–6403, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-251-0. doi: 10.18653/v1/2025.acl-long.320. URL <https://aclanthology.org/2025.acl-long.320/>.
- Pang Wei Koh and Percy Liang. Understanding black-box predictions via influence functions, 2020. URL <https://arxiv.org/abs/1703.04730>.
- Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles, 2017. URL <https://arxiv.org/abs/1612.01474>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, Sedrick Keh, Kushal Arora, Saurabh Garg, Rui Xin, Niklas Muennighoff, Reinhard Heckel, Jean Mercat, Mayee Chen, Suchin Gururangan, Mitchell Wortsman, Alon Albalak, Yonatan Bitton, Marianna Nezhurina, Amro Abbas, Cheng-Yu Hsieh, Dhruva Ghosh, Josh Gardner, Maciej Kilian, Hanlin Zhang, Rulin Shao, Sarah Pratt, Sunny Sanyal, Gabriel Ilharco, Giannis Daras, Kalyani Marathe, Aaron Gokaslan, Jieyu Zhang, Khyathi Chandu, Thao Nguyen, Igor Vasiljevic, Sham Kakade, Shuran Song, Sujay Sanghavi, Fartash Faghri, Sewoong Oh, Luke Zettlemoyer, Kyle Lo, Alaaeldin El-Nouby, Hadi Pouransari, Alexander Toshev, Stephanie Wang, Dirk Groeneveld, Luca Soldaini, Pang Wei Koh, Jenia Jitsev, Thomas Kollar, Alexandros G. Dimakis, Yair Carmon, Achal Dave, Ludwig Schmidt, and Vaishaal Shankar. Datacomp-lm: In search of the next generation of training sets for language models, 2025. URL <https://arxiv.org/abs/2406.11794>.
- Jia LI, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. [<https://huggingface.co/AI-MO/NuminaMath-CoT>] (https://github.com/project-numina/aimo-progress-prize/blob/main/report/numina_dataset.pdf), 2024.
- Xiao Liang, Zhong-Zhi Li, Yeyun Gong, Yang Wang, Hengyuan Zhang, Yelong Shen, Ying Nian Wu, and Weizhu Chen. Sws: Self-aware weakness-driven problem synthesis in reinforcement learning for llm reasoning, 2025. URL <https://arxiv.org/abs/2506.08989>.

- Claudia Malzer and Marcus Baum. A hybrid approach to hierarchical density-based cluster selection. In *2020 IEEE International Conference on Multisensor Fusion and Integration for Intelligent Systems (MFI)*, page 223–228. IEEE, September 2020. doi: 10.1109/mfi49285.2020.9235263. URL <http://dx.doi.org/10.1109/MFI49285.2020.9235263>.
- Shuhaib Mehri, Xiusi Chen, Heng Ji, and Dilek Hakkani-Tür. Beyond sample-level feedback: Using reference-level feedback to guide data synthesis, 2025. URL <https://arxiv.org/abs/2502.04511>.
- Baharan Mirzasoleiman, Jeff Bilmes, and Jure Leskovec. Coresets for data-efficient training of machine learning models, 2020. URL <https://arxiv.org/abs/1906.01827>.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. Mteb: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316*, 2022. doi: 10.48550/ARXIV.2210.07316. URL <https://arxiv.org/abs/2210.07316>.
- Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In Gerardo Flores, George H Chen, Tom Pollard, Joyce C Ho, and Tristan Naumann, editors, *Proceedings of the Conference on Health, Inference, and Learning*, volume 174 of *Proceedings of Machine Learning Research*, pages 248–260. PMLR, 07–08 Apr 2022. URL <https://proceedings.mlr.press/v174/pal22a.html>.
- Guilherme Penedo, Anton Lozhkov, Hynek Kydlíček, Loubna Ben Allal, Edward Beeching, Agustín Piqueres Lajarín, Quentin Gallouédec, Nathan Habib, Lewis Tunstall, and Leandro von Werra. Codeforces cots. <https://huggingface.co/datasets/open-r1/codeforces-cots>, 2025.
- Yuanqian Qi, Xiaohao Yang, Jueqing Lu, Guoxiang Guo, Joanne Enticott, Gang Liu, and Lan Du. Next generation active learning: Mixture of llms in the loop, 2026. URL <https://arxiv.org/abs/2601.15773>.
- Hamidreza Rouzegar and Masoud Makrehchi. Enhancing text classification through LLM-driven active learning and human annotation. In Sophie Henning and Manfred Stede, editors, *Proceedings of the 18th Linguistic Annotation Workshop (LAW-XVIII)*, pages 98–111, St. Julians, Malta, March 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.law-1.10. URL <https://aclanthology.org/2024.law-1.10/>.
- Ozan Sener and Silvio Savarese. Active learning for convolutional neural networks: A core-set approach, 2018. URL <https://arxiv.org/abs/1708.00489>.
- Burr Settles. *Active learning*. Morgan & Claypool Publishers, 2012.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Shivchander Sudalairaj, Abhishek Bhandwaladar, Aldo Pareja, Kai Xu, David D Cox, and Akash Srivastava. Lab: Large-scale alignment for chatbots. *arXiv preprint arXiv:2403.01081*, 2024.
- Qiushi Sun, Jingyang Gong, Qipeng Guo, Lei Li, and Fei Yuan. Codeevo: Interaction-driven synthesis of code-centric data through hybrid and iterative feedback. In *NeurIPS 2025 Fourth Workshop on Deep Learning for Code*, 2025. URL <https://openreview.net/forum?id=hPKoXc68sy>.
- Gaoang Wang, Jenq-Neng Hwang, Craig Rose, and Farron Wallace. Uncertainty sampling based active learning with diversity constraint by sparse selection. In *2017 IEEE 19th International Workshop on Multimedia Signal Processing (MMSP)*, pages 1–6. IEEE, 2017.
- Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. Self-instruct: Aligning language models with self-generated instructions. In Anna Rogers, Jordan Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13484–13508, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.acl-long.754. URL <https://aclanthology.org/2023.acl-long.754/>.

Dataset	SM as Labeler		% Correct		GPT as Labeler	
	Numina	MedMCQA	Numina	MedMCQA	Numina	MedMCQA
Format	29.6	57.3	27.9	58.7	37.7 $\uparrow 8.1$	61.5 $\uparrow 4.2$
Confidence	33.7	59.5	32.5	53.9	36.2 $\uparrow 2.5$	62.6 $\uparrow 3.1$
Proximity	22.1	63.6	77.0	54.3	39.8 $\uparrow 17.7$	67.1 $\uparrow 3.5$
Gradient Norm	29.7	58.6	29.0	58.4	38.6 $\uparrow 8.9$	63.3 $\uparrow 4.7$
Diversity	24.7	58.0	29.6	48.6	38.1 $\uparrow 13.4$	58.1 $\uparrow 0.1$
Answer Variance	30.5	55.1	36.1	65.4	36.7 $\uparrow 6.2$	61.4 $\uparrow 6.3$
Average	28.4	58.7	38.7	56.6	37.9 $\uparrow 9.5$	62.3 $\uparrow 3.7$

Table 6: The upper bound performance of student models trained on AcquisitionSynthesis generated questions. “SM as Labeler” is the performance of the student models when the student model (SM) itself is the labeler for the generated questions. These numbers are the same as those in Table 1. “% Correct” refers to the percent of answers generated by the student model that are correct, according to GPT-4o-mini. Finally, “GPT as Labeler” is the upper bound performance – the performance of students models when GPT-4o-mini is the labeler.

Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. LESS: Selecting influential data for targeted instruction tuning. In *International Conference on Machine Learning (ICML)*, 2024.

Can Xu, Qingfeng Sun, Kai Zheng, Xiubo Geng, Pu Zhao, Jiazhan Feng, Chongyang Tao, Qingwei Lin, and Daxin Jiang. Wizardlm: Empowering large pre-trained language models to follow complex instructions, 2025. URL <https://arxiv.org/abs/2304.12244>.

Tianhui Zhang, Bei Peng, and Danushka Bollegala. Synthetic data generation for training diversified commonsense reasoning models, 2026. URL <https://arxiv.org/abs/2603.18361>.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*, 2025.

A The Upper Bound: using GPT as a labeler

In this appendix, we introduce RQ5: how do models trained with AcquisitionSynthesis data compare to models trained with data distilled from proprietary models?

We break this experiment into two questions: (a) how accurate are the answers generated from the AcquisitionSynthesis models? and (b) what would be the difference if the answers were instead generated by oracle, how accurate would student models be? This experiment is to showcase the upper bound of performance we can get from student models trained with AcquisitionSynthesis models. To test (a), we ask GPT-4o-mini whether the generated answer from AcquisitionSynthesis is correct or not. According to the “% Correct” column in Table 6, an average of 52.4% of answers are incorrect. To answer how this affects the answer of the student model (b), we use the questions generated by AcquisitionSynthesis and answer them using GPT-4o-mini, and train the student model again (similar to setup for RQ1). According to the “GPT as Labeler” column in Table 6, student models achieve an average 6.6% boost in performance with more accurate labels.

B Details of AcquisitionSynthesis training

In this section, we elaborate on the AcquisitionSynthesis training. We use 500 training samples from our selected benchmark during GRPO training. We give these 500 training samples as in-context learning samples. The reason why we use 500 samples is to avoid reward hacking. We saw that with 2,000 training samples, the trained AcquisitionSynthesis model generated the exact same training sample every time – this is because the model reward-hacked, and found one sample that maximized

the acquisition reward (or at least, one peak of it). We see that 500 samples is a good balance between learning to maximize the acquisition reward and reward hacking.

Our training prompts for Numina and MedMCQA are in Figures 4 and 5, respectively.

Prompt for AcquisitionSynthesis training on Numina

Generate a NEW, ORIGINAL math problem that is AS DIFFICULT than the reference below. Do NOT copy, paraphrase, or reuse it in any way.

Reference (difficulty calibration only - do not reproduce):

<question>
{question}
</question>

<reasoning>
{reasoning}
</reasoning>

<answer>
{answer}
</answer>

Requirements for your generated problem:

- Requires non-trivial reasoning steps (no single-step shortcuts)
- Draws from: number theory, combinatorics, algebra, geometry, or probability
- Is self-contained and precisely stated
- Reasoning includes a complete step-by-step derivation
- Answer includes just the final result

IMPORTANT: generate a (question, reasoning, answer) triplet; wrap your question, reasoning, and answer in the following special tokens:

<question> Insert your question here. </question>
<reasoning> Insert the thinking and general reasoning here. </reasoning>
<answer> Insert your short answer here. </answer>

Figure 4: Prompt used during AcquisitionSynthesis training. As input, the prompt takes a question, answer, and reasoning from the Numina training dataset.

Prompt for AcquisitionSynthesis training on Numina

Generate a NEW, ORIGINAL medical multiple-choice question that is AS DIFFICULT as the reference below. Do NOT copy, paraphrase, or reuse it in any way.

Reference (difficulty calibration only - do not reproduce):

```
<question>
{question}
</question>
```

```
<reasoning>
{reasoning}
</reasoning>
```

```
<answer>
{answer}
</answer>
```

Requirements for your generated problem:

- Requires non-trivial clinical or biomedical reasoning (no single-step lookups)
- Draws from: pharmacology, pathophysiology, clinical medicine, anatomy, or biochemistry
- Includes exactly 4 answer choices labeled A, B, C, D - only one is correct
- Plausible distractors that reflect common misconceptions or near-miss diagnoses
- Is self-contained and precisely stated
- Reasoning includes a complete step-by-step clinical justification for the correct choice and why distractors are wrong
- Answer includes the correct letter and the full text of the chosen option

IMPORTANT: generate a (question, reasoning, answer) triplet; wrap your question, reasoning, and answer in the following special tokens:

```
<question> Insert your question and answer choices
(A/B/C/D) here. </question>
```

```
<reasoning> Insert the step-by-step clinical reasoning here, including
why each distractor is incorrect. </reasoning>
```

```
<answer> Insert the correct letter and its full answer text, e.g.
"A: Metformin". </answer>
```

Figure 5: Prompt used during AcquisitionSynthesis training. As input, the prompt takes a question, answer, and reasoning from the MedMCQA training dataset.